

Paso 13 - Todo junto | Solucionario

Las cinco soluciones del paso 13. Los cinco ejercicios se hacen sobre **el mismo programa**, así que la solución completa está en el B13.5: ahí está la calculadora con todo aplicado, y los cuatro anteriores muestran solo el pedazo que les toca.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso13` y `javac *.java`.

B13.1 - El alumno que aprobó y reprobó a la vez

El promedio de 3.9 y 4.0 es **3.95**. `printf("%.1f")` lo **muestra** como 4.0, pero el `if` compara el valor de verdad, que es menor que 4.0. El programa nunca estuvo mal: mostraba una cosa y comparaba otra.

Antes y después:

```
ANTES (sin redondear)
    Promedio: 4.0
    REPROBADO, le faltan 0.0

DESPUES (con Notas.redondear)
    Promedio: 4.0
    APROBADO
```

El arreglo es un método nuevo en `Notas`:

```
public static double redondear(double valor) {
    return Math.round(valor * 10) / 10.0;
}
```

`Math.round(3.95 * 10)` da 40, y `40 / 10.0` da 4.0. **El 10.0 es obligatorio:** con `10` a secas sería una división entera (paso 3) y todos los promedios quedarían sin decimales.

¿Dónde va? En `promedio`, no en `informe`. Razón: el redondeo a un decimal es parte de **qué es** un promedio de notas en DUOC, no de cómo se muestra. Si lo pones solo en el informe, el día que alguien use `Notas.promedio` para otra cosa —un ranking, un archivo— el 3.95 vuelve a aparecer y el bug renace. **La regla arreglada vive en la clase, no en la pantalla.**

Conviene decirlo en voz alta: este ejercicio no tenía ningún error de programación. El código hacía exactamente lo que decía. Lo que estaba mal era **la regla**, y esos son los bugs que llegan a producción.

B13.2 - El informe completo

Tres datos más en el recuadro, **sin un solo** `for` **nuevo**: los tres métodos ya existen en `Notas`.

Salida:

```
===== Ana Perez =====
Nota 1: 5.5
Nota 2: 6.2
Nota 3: 3.1
Promedio: 4.9
Maxima: 6.2   Minima: 3.1   Rojas: 1
APROBADO
```

Lo que importa. La línea nueva es un `printf` con tres llamadas:

```
System.out.printf("  Maxima: %.1f   Minima: %.1f   Rojas: %d%n",
                  Notas.maxima(notas), Notas.minima(notas), Notas.contarRojas(notas));
```

Tres recorridos del arreglo escritos como tres palabras. Ese es el pago de haber escrito la clase `Notas` en el paso 10: el programa que la usa **describe lo que quiere**, no cómo se calcula.

Si tuviste que escribir un `for` aquí, es porque te faltaba `minima`: se agregó en el B10.1 y son cuatro líneas, calcadas de `maxima` cambiando el `>` por un `<`.

B13.3 - El resumen del curso

Cuatro acumuladores declarados **antes** del ciclo, que crecen en cada vuelta.

Salida:

```
===== RESUMEN DEL CURSO =====
```

```
Alumnos ingresados: 3
```

```
Aprobados: 2 (66.7%)
```

```
Reprobados: 1
```

```
Mejor promedio: 4.9 (Ana Perez)
```

Lo que importa: dónde se declara cada cosa. `cuantos` y `aprobados` van **antes** del ciclo; si los declaras adentro, vuelven a nacer en 0 en cada alumno y el resumen siempre dice 1. Es el mismo error que el `boolean esPrimo` del B7.5, al revés.

El `100.0` otra vez: `aprobados * 100 / cuantos` con enteros daría 66 en vez de 66.7.

Los reprobados no se cuentan, se restan: `cuantos - aprobados`. Un dato que se puede calcular a partir de otros no se guarda; guardarlo es una oportunidad más de que queden descuadrados.

El mejor promedio usa el truco del paso 8, con un detalle: la condición es `cuantos == 1 || promedio > mejorPromedio`. No se puede partir suponiendo que el mejor es 0, porque si **todos** reprobaban con notas bajo... bueno, 0 no es una nota posible y funcionaría igual. Pero la costumbre correcta es la de siempre: **el primero es el mejor hasta que aparezca otro**, y así el código sigue funcionando aunque el rango cambie.

Fíjate en que hay que guardar **dos** cosas, el promedio y el nombre, y mantenerlas sincronizadas a mano. Es el mismo problema de los arreglos paralelos del B13.5, en chiquitito.

B13.4 - El ramo con ponderaciones

Una pregunta antes de las notas y, según la respuesta, el `promedio` simple o el sobrecargado del B12.3.

Salida:

```
Nombre del alumno: Luis Soto
Ponderado? (s/n): s
Nota 1 (30%): 5.0
Nota 2 (30%): 6.0
Nota 3 (40%): 4.0

===== Luis Soto =====
Nota 1: 5.0
Nota 2: 6.0
Nota 3: 4.0
Promedio ponderado: 4.9
Maxima: 6.0   Minima: 4.0   Rojas: 0
APROBADO
```

Lo que importa. El `if (ponderado)` decide **dos** cosas a la vez: cuántas notas se piden (tres fijas contra las que diga el usuario) y **cuál de las dos versiones de `Notas.promedio` se llama**. Java elige la versión por los parámetros, tal como el paso 12.

Los pesos son una constante de la clase:

```
static final double[] PESOS = {0.3, 0.3, 0.4};
```

Así el 30/30/40 aparece **una vez**: se usa para saber cuántas notas pedir (`PESOS.length`), para escribir el porcentaje en la pregunta (`(int) (PESOS[i] * 100)`) y para calcular. Cambiar la ponderación a 20/40/40 es cambiar esa línea y nada más.

El `ponderado ? " ponderado" : ""` del `printf` es el ternario del B8.5: la misma línea sirve para los dos casos y escribe la palabra solo cuando corresponde.

B13.5 - Desafío final: el curso completo

Dos arreglos paralelos (`nombres` y `promedios`) y una **selección directa**: buscar el mejor de los que quedan, mostrarlo, marcarlo, repetir.

▼ Ver la solución (Notas.java, Validador.java, B13_Calculadora.java)

Notas.java

```

/*
 * NOTAS - la version final, con todo lo que le agregaron los ejercicios:
 * minima (B10.1), el promedio ponderado (B12.3) y redondear (B13.1).
 */
public class Notas {

    public static final double NOTA_MINIMA = 1.0;
    public static final double NOTA_MAXIMA = 7.0;
    public static final double NOTA_APROBACION = 4.0;

    /** Deja el valor con UN decimal: 3.95 -> 4.0 */
    public static double redondear(double valor) {
        return Math.round(valor * 10) / 10.0;
    }

    public static double promedio(double[] notas) {
        double suma = 0;
        for (double n : notas) {
            suma += n;
        }
        return redondear(suma / notas.length);
    }

    public static double promedio(double[] notas, double[] pesos) {
        double suma = 0;
        for (int i = 0; i < notas.length; i++) {
            suma += notas[i] * pesos[i];
        }
        return redondear(suma);
    }

    public static double maxima(double[] notas) {
        double mayor = notas[0];
        for (double n : notas) {
            if (n > mayor) {
                mayor = n;
            }
        }
        return mayor;
    }

    public static double minima(double[] notas) {

```

```
        double menor = notas[0];
        for (double n : notas) {
            if (n < menor) {
                menor = n;
            }
        }
        return menor;
    }

    public static int contarRojas(double[] notas) {
        int rojas = 0;
        for (double n : notas) {
            if (n < NOTA_APROBACION) {
                rojas++;
            }
        }
        return rojas;
    }

    public static boolean aprobo(double promedio) {
        return promedio >= NOTA_APROBACION;
    }
}
```

Validador.java

```

import java.util.Scanner;

/*
 * VALIDADOR - el del paso 11, con los tres metodos de la hoja agregados.
 *
 * es... / esta... PREGUNTAN: devuelven true o false.
 * leer... INSISTEN: piden hasta que el dato sirva.
 */
public class Validador {

    // ----- los que PREGUNTAN -----

    public static boolean estaEnRango(double valor, double min, double max) {
        return valor >= min && valor <= max;
    }

    public static boolean esCorreoDuoc(String correo) {
        String dominio = "@duocuc.cl";
        return correo.endsWith(dominio)
            && correo.length() > dominio.length()
            && !correo.contains(" ");
    }

    // B11.2
    public static boolean esClaveSegura(String clave) {
        if (clave.length() < 8) {
            return false;
        }
        boolean tieneDigito = false;
        boolean tieneMayuscula = false;
        for (int i = 0; i < clave.length(); i++) {
            char c = clave.charAt(i);
            if (Character.isDigit(c)) {
                tieneDigito = true;
            }
            if (Character.isUpperCase(c)) {
                tieneMayuscula = true;
            }
        }
        return tieneDigito && tieneMayuscula;
    }
}

```

```

// B11.4
public static boolean esRutValido(String rut) {
    int guion = rut.indexOf('-');
    if (guion < 7 || guion != rut.lastIndexOf('-')) {
        return false; // sin guion, muy corto, o con
dos
    }
    if (rut.length() != guion + 2) {
        return false; // despues del guion va UN
caracter
    }
    for (int i = 0; i < guion; i++) {
        if (!Character.isDigit(rut.charAt(i))) {
            return false;
        }
    }
    char dv = rut.charAt(guion + 1);
    return Character.isDigit(dv) || dv == 'k' || dv == 'K';
}

// B11.4 extra: el digito verificador de verdad (modulo 11)
public static boolean tieneDvCorrecto(String rut) {
    if (!esRutValido(rut)) {
        return false;
    }
    int guion = rut.indexOf('-');
    String cuerpo = rut.substring(0, guion);

    int suma = 0;
    int multiplicador = 2;
    for (int i = cuerpo.length() - 1; i >= 0; i--) { // de derecha a
izquierda
        suma += (cuerpo.charAt(i) - '0') * multiplicador;
        multiplicador++;
        if (multiplicador > 7) {
            multiplicador = 2;
        }
    }

    int resto = 11 - (suma % 11);
    char esperado;
    if (resto == 11) {
        esperado = '0';
    }

```

```

        } else if (resto == 10) {
            esperado = 'K';
        } else {
            esperado = (char) ('0' + resto);
        }

        return Character.toUpperCase(rut.charAt(guion + 1)) == esperado;
    }

    // ----- los que INSISTEN -----

    public static String leerTexto(Scanner sc, String pregunta) {
        System.out.print(pregunta);
        String texto = sc.nextLine().trim();
        while (texto.isEmpty()) {
            System.out.println("    No puede quedar vacio.");
            System.out.print(pregunta);
            texto = sc.nextLine().trim();
        }
        return texto;
    }

    public static int leerEntero(Scanner sc, String pregunta, int min, int max)
    {
        while (true) {
            System.out.print(pregunta);
            if (!sc.hasNextInt()) {
                System.out.println("    Eso no es un numero entero.");
                sc.nextLine();
            } else {
                int valor = sc.nextInt();
                sc.nextLine();
                if (estaEnRango(valor, min, max)) {
                    return valor;
                }
                System.out.println("    Tiene que ir de " + min + " a " + max +
".");
            }
        }
    }

    public static double leerDecimal(Scanner sc, String pregunta, double min,
double max) {

```

```

        while (true) {
            System.out.print(pregunta);
            if (!sc.hasNextDouble()) {
                System.out.println("    Eso no es un numero.");
                sc.nextLine();
            } else {
                double valor = sc.nextDouble();
                sc.nextLine();
                if (estaEnRango(valor, min, max)) {
                    return valor;
                }
                System.out.println("    Tiene que ir de " + min + " a " + max +
".");
            }
        }
    }

// B11.1
public static boolean leerSiNo(Scanner sc, String pregunta) {
    while (true) {
        String respuesta = leerTexto(sc, pregunta);
        if (respuesta.equalsIgnoreCase("s")) {
            return true;
        }
        if (respuesta.equalsIgnoreCase("n")) {
            return false;
        }
        System.out.println("    Responde s o n.");
    }
}
}

```

B13_Calculadora.java

```

import java.util.Scanner;

/*
 * PASO 13 - LA CALCULADORA DE NOTAS, CON LOS CINCO EJERCICIOS RESUELTOS.
 *
 * B13.1 el promedio se redondea en Notas.redondear
 * B13.2 el informe muestra maxima, minima y rojas
 * B13.3 el resumen del curso al salir
 * B13.4 promedio simple o ponderado
 * B13.5 el listado final ordenado de mayor a menor
 *
 * javac Notas.java Validador.java B13_Calculadora.java
 * java B13_Calculadora
 */
public class B13_Calculadora {

    static final int MAXIMO_ALUMNOS = 50;
    static final double[] PESOS = {0.3, 0.3, 0.4};

    static void informe(String nombre, double[] notas, double promedio, boolean
ponderado) {
        System.out.println();
        System.out.println("==== " + nombre + " =====");
        for (int i = 0; i < notas.length; i++) {
            System.out.printf(" Nota %d: %.1f%n", i + 1, notas[i]);
        }

        System.out.printf(" Promedio%s: %.1f%n", ponderado ? " ponderado" :
"", promedio);
        System.out.printf(" Maxima: %.1f Minima: %.1f Rojas: %d%n",
            Notas.maxima(notas), Notas.minima(notas),
            Notas.contarRojas(notas));

        if (Notas.aprobo(promedio)) {
            System.out.println(" APROBADO");
        } else {
            System.out.printf(" REPROBADO, le faltan %.1f%n",
                Notas.NOTA_APROBACION - promedio);
        }
        System.out.println();
    }
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    String[] nombres = new String[MAXIMO_ALUMNOS];
    double[] promedios = new double[MAXIMO_ALUMNOS];
    int cuantos = 0;
    int aprobados = 0;
    double mejorPromedio = 0;
    String mejorAlumno = "";

    boolean seguir = true;
    while (seguir && cuantos < MAXIMO_ALUMNOS) {
        String nombre = Validador.leerTexto(sc, "Nombre del alumno: ");

        boolean ponderado = Validador.leerSiNo(sc, "Ponderado? (s/n): ");
        double[] notas;
        double promedio;

        if (ponderado) {
            notas = new double[PESOS.length];
            for (int i = 0; i < notas.length; i++) {
                notas[i] = Validador.leerDecimal(sc,
                    "Nota " + (i + 1) + " (" + (int) (PESOS[i] * 100) +
"%): ",
                    Notas.NOTA_MINIMA, Notas.NOTA_MAXIMA);
            }
            promedio = Notas.promedio(notas, PESOS);
        } else {
            int cantidad = Validador.leerEntero(sc, "Cuantas notas tiene?
", 1, 10);

            notas = new double[cantidad];
            for (int i = 0; i < notas.length; i++) {
                notas[i] = Validador.leerDecimal(sc, "Nota " + (i + 1) + ":
",
                    Notas.NOTA_MINIMA, Notas.NOTA_MAXIMA);
            }
            promedio = Notas.promedio(notas);
        }

        informe(nombre, notas, promedio, ponderado);

        nombres[cuantos] = nombre;
        promedios[cuantos] = promedio;
    }
}

```

```

        cuantos++;
        if (Notas.aprobo(promedio)) {
            aprobados++;
        }
        if (cuantos == 1 || promedio > mejorPromedio) { // el primero
manda, despues gana el mayor
            mejorPromedio = promedio;
            mejorAlumno = nombre;
        }

        seguir = Validador.leerSiNo(sc, "Otro alumno? (s/n): ");
    }

    // ----- B13.3: el resumen -----
    System.out.println();
    System.out.println("==== RESUMEN DEL CURSO =====");
    System.out.println("Alumnos ingresados: " + cuantos);
    System.out.printf("Aprobados: %d (%.1f%%)\n", aprobados, aprobados *
100.0 / cuantos);
    System.out.println("Reprobados: " + (cuantos - aprobados));
    System.out.printf("Mejor promedio: %.1f (%s)\n", mejorPromedio,
mejorAlumno);

    // ----- B13.5: el listado ordenado -----
    System.out.println();
    System.out.println("==== NOTAS FINALES =====");
    boolean[] yaMostrado = new boolean[cuantos];

    for (int puesto = 1; puesto <= cuantos; puesto++) {
        int mejor = -1;
        for (int i = 0; i < cuantos; i++) {
            if (!yaMostrado[i] && (mejor == -1 || promedios[i] >
promedios[mejor])) {
                mejor = i;
            }
        }
        yaMostrado[mejor] = true;
        System.out.printf("%d. %-14s %.1f %s\n", puesto, nombres[mejor],
promedios[mejor],
            Notas.aprobo(promedios[mejor]) ? "APROBADO" : "REPROBADO");
    }

    System.out.println();

```

```
        System.out.println("Listo.");  
        sc.close();  
    }  
}
```

Salida:

Nombre del alumno: Ana Perez

Ponderado? (s/n): n

Cuantas notas tiene? 3

Nota 1: 5.5

Nota 2: 6.2

Nota 3: 3.1

===== Ana Perez =====

Nota 1: 5.5

Nota 2: 6.2

Nota 3: 3.1

Promedio: 4.9

Maxima: 6.2 Minima: 3.1 Rojas: 1

APROBADO

Otro alumno? (s/n): s

Nombre del alumno: Luis Soto

Ponderado? (s/n): s

Nota 1 (30%): 5.0

Nota 2 (30%): 6.0

Nota 3 (40%): 4.0

===== Luis Soto =====

Nota 1: 5.0

Nota 2: 6.0

Nota 3: 4.0

Promedio ponderado: 4.9

Maxima: 6.0 Minima: 4.0 Rojas: 0

APROBADO

Otro alumno? (s/n): s

Nombre del alumno: Pedro Diaz

Ponderado? (s/n): n

Cuantas notas tiene? 2

Nota 1: 3.5

Nota 2: 3.9

===== Pedro Diaz =====

Nota 1: 3.5

Nota 2: 3.9

Promedio: 3.7

Maxima: 3.9 Minima: 3.5 Rojas: 2

```
REPROBADO, le faltan 0.3

Otro alumno? (s/n): n

===== RESUMEN DEL CURSO =====
Alumnos ingresados: 3
Aprobados: 2 (66.7%)
Reprobados: 1
Mejor promedio: 4.9 (Ana Perez)

===== NOTAS FINALES =====
1. Ana Perez      4.9  APROBADO
2. Luis Soto      4.9  APROBADO
3. Pedro Diaz     3.7  REPROBADO

Listo.
```

El algoritmo del ordenamiento (se llama *selección*): un arreglo `boolean[] yaMostrado` marca a los que ya salieron; en cada vuelta se busca el mayor entre los no marcados, se imprime y se marca. Son dos `for` anidados y no necesita ordenar de verdad el arreglo.

Los arreglos paralelos son el problema. `nombres[i]` y `promedios[i]` son el mismo alumno **solo porque tú los mantienes sincronizados a mano**. Si mañana agregas el RUT, son tres arreglos; si ordenas uno y te olvidas del otro, los promedios quedan con el dueño equivocado y **nadie te avisa**: compila igual y el informe miente.

Y esa es la pregunta de cierre de los 13 pasos. Lo que falta es poder decir "un alumno" y que el nombre, las notas y el promedio viajen **juntos** en una sola cosa. Esa cosa se llama **objeto**, y es la [demo 1](#):

```
Alumno ana = new Alumno("Ana Perez");
ana.agregarNota(5.5);
System.out.println(ana.getPromedio());
```

Un arreglo `Alumno[] curso` no se puede desincronizar: cada casillero trae al alumno entero. Con eso, el B13.5 se escribe en la mitad de líneas y sin `yaMostrado`.

Volver a la [hoja del paso 13](#) o al [paso 13 de la guía](#).